

 **MLOPS RISK**

Feedback Loop Contamination

When your AI learns from its own outputs, it collapses. When your training data leaks into your eval set, your metrics lie. Feedback contamination is one of the most silent model killers in production.

6 contamination sources covered in this carousel



MODEL COLLAPSE



EVAL LEAKAGE



SELECTION BIAS



KAPPA AGREEMENT

Production-grade detection code and prevention strategies.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**

What Is Feedback Loop Contamination?

Feedback loop contamination happens when the outputs of your AI system become inputs to the training or evaluation process, creating a closed loop where errors reinforce themselves until the model collapses or metrics stop reflecting reality.

💀 Model Collapse

The AI generates text. Users rate it. Those rated outputs get used for fine-tuning. The model now learns its own biases as ground truth. Over time, diversity drops to near zero and the model repeats a narrow set of safe-sounding but useless responses.

🔍 Eval Set Leakage

Training examples that are too similar to evaluation examples cause artificially inflated RAGAS, BLEU, and accuracy scores. Your eval claims 92% accuracy. Real users see 60% quality. The gap is contamination.

📉 The Silent Progression

Quality does not crash immediately. It degrades linearly over 3-6 fine-tuning cycles, making it nearly impossible to notice without explicit diversity and agreement monitoring.



6 Contamination Sources in Production

Source	How It Happens	Detection Signal
Model collapse	Fine-tune on own outputs	Diversity score below 0.30
Eval leakage	Train/eval overlap	Cosine sim above 0.90
Annotator bias	Single human labels all data	Cohen's Kappa below 0.60
Selection bias	Only active users give feedback	Feedback rate by quartile diverges
Confirmation bias	Teams label to confirm model	Label flips below 5% per cycle
Temporal leakage	Future data in past eval set	Eval date before training cutoff



Every one of these sources produces inflated metrics and degraded real-world performance. Monitor all six simultaneously, not just the one you already know about.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Measuring Output Diversity Score

A drop in pairwise cosine diversity across model outputs is the earliest signal of model collapse. Track this per fine-tuning cycle.

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def compute_diversity_score(embeddings: list) → float:
    """Pairwise avg cosine distance. Higher = more diverse."""
    if len(embeddings) < 2:
        return 1.0
    mat = np.array(embeddings)
    sim = cosine_similarity(mat) # NxN similarity matrix
    n = len(sim)
    upper = [sim[i][j] for i in range(n)
              for j in range(i + 1, n)]
    avg_sim = np.mean(upper)
    return round(1.0 - float(avg_sim), 4)

# Weekly check: below 0.30 = model collapse risk
diversity = compute_diversity_score(production_embeddings)
if diversity < 0.30:
    alert(f"Collapse risk: diversity={diversity:.3f}")
```



Run weekly on 200 randomly sampled production outputs. Plot as a time series. A downward trend over 3 cycles is an early collapse signal, even before human reviewers notice it.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

Cohen's Kappa Annotator Agreement

Kappa measures inter-annotator agreement adjusted for chance. A single annotator labeling all data has Kappa = 1 by construction but zero reliability. Require two independent annotators and track Kappa over time.

```
from sklearn.metrics import cohen_kappa_score
import numpy as np

def measure_annotator_agreement(
    annotators: list[list[int]]
) → dict:
    """All pairwise kappa between N annotators."""
    n = len(annotators)
    scores = []
    for i in range(n):
        for j in range(i + 1, n):
            k = cohen_kappa_score(annotators[i], annotators[j])
            scores.append(k)
    avg_kappa = float(np.mean(scores))
    if avg_kappa < 0.60:
        alert("Low agreement: audit guidelines before training")
    return {"avg_kappa": avg_kappa, "n_pairs": len(scores)}
```

✓ Good: Kappa above 0.80

Strong agreement. Labels are reliable for fine-tuning and RLHF.

✗ Bad: Kappa below 0.60

Labels are noisy. Training on them injects bias and instability.



Multi-Model Ensemble Labeling

Instead of a single model-as-judge, use two different LLMs as co-annotators. Only accept labels where both agree. This reduces both model-specific and human annotator bias.

```
async def ensemble_label(question: str, answer: str) → dict:
    # Two independent judges (different providers)
    gpt_score, claude_score = await asyncio.gather(
        judge_with_gpt4o(question, answer),
        judge_with_claude(question, answer)
    )
    if abs(gpt_score - claude_score) ≤ 1:
        return {
            "label": round((gpt_score + claude_score) / 2),
            "agreement": "high",
            "use_for_training": True
        }
    else:
        return {
            "label": None,
            "agreement": "low",
            "use_for_training": False
        }
    # Disagreement items route to human review queue
```



Discard labels where the two judges disagree by more than 1 point on a 5-point scale. Only agreed labels enter fine-tuning. This removes 15-25% of labels but dramatically reduces training noise.



AI Coach John

Follow



Repost

Detecting Selection Bias in Feedback

Power users (top 10% by activity) give 60-80% of all feedback. Their preferences are systematically different from casual users. Training on unweighted feedback optimizes for power users and hurts everyone else.

```
import pandas as pd

def detect_selection_bias(df: pd.DataFrame) → dict:
    # df columns: user_id, user_activity_decile
    counts_by_decile = df.groupby("user_activity_decile")["user_id"].count()
    total = counts_by_decile.sum()
    pcts = (counts_by_decile / total * 100).round(1)

    top10_share = pcts.get(10, 0)
    bot10_share = pcts.get(1, 0)
    bias_ratio = top10_share / max(bot10_share, 0.1)

    if bias_ratio > 5.0:
        alert(f"Selection bias: top users give {bias_ratio:.1f}x more")
    return {"bias_ratio": round(bias_ratio, 2), "pcts": pcts}
```

Fix: Inverse Propensity Weighting

Weight each feedback item by $1 / \text{feedback_rate_for_user_segment}$. Users who rarely give feedback have their rare feedback weighted higher. Restores representativeness without discarding data.



Protected Eval Set with Integrity Check

The evaluation set must never change without explicit approval. Any undetected mutation, even adding one new example that resembles training data, invalidates all historical comparisons.

```
import hashlib, json, time

class ProtectedEvalSet:
    def __init__(self, examples: list[dict]):
        self.examples = examples
        self.hash      = self._compute_hash(examples)
        self.created   = time.time()

    def _compute_hash(self, ex) → str:
        blob = json.dumps(ex, sort_keys=True).encode()
        return hashlib.sha256(blob).hexdigest()

    def verify_integrity(self) → bool:
        current = self._compute_hash(self.examples)
        ok = current == self.hash
        if not ok:
            alert("CRITICAL: Eval set hash mismatch. "
                  "Metrics unreliable until resolved.")
        return ok
```

 Store the eval set hash in CI/CD. Any PR that changes eval set content triggers a mandatory review gate. No silent modifications to the golden benchmark, ever.



AI Coach John

Follow



 Repost

Training Data Lineage Tracking

Every training example should carry metadata about its origin: human-written, AI-generated, or AI-then-human-revised. Training on unlabeled AI-generated text is the most common path to model collapse.

```
from dataclasses import dataclass, field
from enum import Enum

class Source(Enum):
    HUMAN = "human"
    AI_GENERATED = "ai_generated"
    AI_REVISIED = "ai_human_revised"
    SYNTHETIC = "synthetic"

@dataclass
class TrainingExample:
    question: str
    answer: str
    source: Source
    model_ver: str = ""
    reviewer: str = ""
    created_ts: float = field(default_factory=time.time)

def check_ai_ratio(examples: list[TrainingExample]) → float:
    ai = sum(1 for e in examples if e.source == Source.AI_GENERATED)
    ratio = ai / len(examples)
```



Rule of Thumb

Keep AI-generated content below 30% of any fine-tuning batch. Pure AI-generated with no human review must not exceed 10%.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

Contamination Monitoring Runbook

Check	Frequency	Alert Threshold	Action
Output diversity score	Weekly	Below 0.30	Pause fine-tuning
Annotator Kappa	Per labeling batch	Below 0.60	Audit guidelines
Train/eval overlap	Before every eval run	Cosine sim above 0.90	Rebuild eval set
Selection bias ratio	Monthly	Above 5x	Apply IPW weights
AI content in training	Per fine-tune job	Above 30%	Add human review
Eval set hash	Every CI run	Any mismatch	Block deploy

Who Owns This?

All checks as automated MLOps pipeline jobs. Slack alerts go to the model training lead. Eval set hash check runs in every GitHub Actions CI workflow before any deploy.



AI Coach John

Follow



 Repost

Clean Feedback Loop Architecture

The safest architecture separates data flows completely: production outputs go to a staging area, not directly into training. A dedicated data quality pipeline validates, deduplicates, and lineage-tags every item before it touches training.

✓ Clean Architecture: 4 Steps

1. Production outputs land in a raw feedback store (append-only, never read directly for training).
2. Daily pipeline runs diversity scoring, Kappa checks, and AI-ratio checks on the raw store.
3. Items passing all checks move to the clean training pool with lineage metadata attached.
4. Fine-tuning jobs consume only from the clean pool, never from raw.

⚖️ Eval Set Rules

Eval set lives in a separate read-only repository with a protected main branch. Any proposed change requires 2 human approvers and triggers a full re-run of all historical benchmarks before merge.



Feedback Contamination Mistakes

- ✖ **Fine-tuning on raw production outputs without human review.** The model learns its own hallucinations as facts. Quality degrades across 3-5 fine-tuning cycles before becoming noticeable to human reviewers.
- ✖ **Using the same dataset for training and evaluation.** Even partial overlap causes RAGAS and accuracy scores to be 10-30% higher than real-world performance.
- ✖ **Single annotator labeling all data.** Kappa cannot be computed. That person's bias gets baked into the entire fine-tuned model permanently.
- ✖ **Never checking output diversity.** Model collapse is invisible in accuracy metrics. Pairwise cosine diversity is a separate required metric.
- ✖ **Not tracking source of each training example.** Unlabeled AI-generated content silently accumulates and exceeds the safe threshold before anyone notices.
- ✖ **Mutable eval sets without version control.** A team member adds examples to make scores look better. All historical comparisons become meaningless immediately.





Feedback Contamination Full Reference

Signal	Code / Metric	Safe Threshold
Output diversity	<code>compute_diversity_score()</code>	Above 0.30
Annotator agreement	<code>cohen_kappa_score(a, b)</code>	Above 0.60
Eval set integrity	SHA-256 hash check in CI	Zero mismatches
AI content ratio	<code>TrainingExample.source</code> check	Below 30%
Selection bias	Feedback rate by user decile	Ratio below 5x
Train/eval overlap	Pairwise cosine similarity	Max sim below 0.90
Label agreement	<code>ensemble_label()</code> two-model vote	Disagree: discard



Clean data in, clean model out. Contaminated feedback in, model collapse out.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ ↻ Repost

 KEY TAKEAWAY

Contamination Is Silent Until It Is Catastrophic

No alarm fires when model collapse begins. Metrics drift slowly upward on your inflated eval set while real-world quality quietly degrades. The only defense is systematic monitoring before the damage compounds.

Three Non-Negotiables

- Weekly output diversity scoring. Below 0.30 means pause all fine-tuning immediately.
- Protected eval set with SHA-256 hash verified in every CI run. No exceptions.
- Two-annotator labeling with Cohen's Kappa above 0.60 before any batch enters training.

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 **Repost**